

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 11

Recap and outline

- Feature selection methods: filter, wrapper, embedded
- Summary of methods
- How do we compare methods?
- Overview of validation approaches

Overview of methods

$$\widehat{S}^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, |S| \leq s} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

- Filter: correlation, mutual information, conditional correlation
 - ▶ Independent of prediction function class /algorithm $\mathcal{F}$
 - ▶ Lose information about class $\mathcal{F}$
- Embedded: OLS+p-values, Lasso, MDI
 - ▶ Do not need to optimize over subsets as it comes free with prediction
 - ▶ Only applies to scenarios where parameters associated with prediction function contain feature selection information
- Wrapper: LOCO, backward selection, forward selection (AIC or BIC)
 - ▶ Need to optimize over both leading to significant computational challenges
 - ▶ Applies to any function class $\mathcal{F}$

$$S^* \in \arg \min_{S \subseteq \{1,2,\dots,p\}, Y \perp X_{S^c} | X_S} |S|$$

$$Y \perp X_j | X_{-j}, \text{ for all } j \in \tilde{S}^c$$

$$Y \not\perp X_j | X_{-j}, \text{ for all } j \in \tilde{S}$$

- **Computational:** Can we compute $\hat{S}^*$ in reasonable time?
- **Statistical:** How close is $\hat{S}^*$ to S^* (or $\tilde{S}$)?

Correlation screening

- **Pros**
- **Cons**

Correlation screening

- **Pros**

- ▶ Fast, simple and interpretable
- ▶ Naturally fits into hypothesis testing framework for feature selection

- **Cons**

- ▶ Captures only linear relationships ($Y = X^2$ for example)
- ▶ Captures marginal dependence rather than conditional dependence so does not learn $\tilde{S}$

Mutual information screening

- **Pros**
- **Cons**

Mutual information screening

- **Pros**

- ▶ Captures all linear and non-linear relationships
- ▶ Mathematically equivalent to capturing dependence/independence

- **Cons**

- ▶ Extremely data and computationally intensive to estimate mutual information
- ▶ Captures marginal dependence rather than conditional dependence so does not estimate $\tilde{S}$

Conditional correlation/OLS

- **Pros**
- **Cons**

- **Pros**

- ▶ Captures conditional relationships matching feature selection and $\tilde{S}$
- ▶ Fast and interpretable
- ▶ Fits into hypothesis testing framework for feature selection

- **Cons**

- ▶ Only captures linear relationships
- ▶ Not naturally able to deal with scenarios when p is large compared to n

Lasso

- **Pros**
- **Cons**

- **Pros**

- ▶ Captures conditional relationships matching feature selection and $\tilde{S}$
- ▶ Reasonably fast and scalable
- ▶ Able to deal with scenarios when p is large compared to n

- **Cons**

- ▶ Need to select regularization parameter λ
- ▶ Again captures only linear relationships
- ▶ Can often be unstable when there are dependent features

MDI (Maximum decrease in impurity)

- **Pros**
- **Cons**

MDI (Maximum decrease in impurity)

- **Pros**

- ▶ Captures non-linear dependence
- ▶ Usually gives good prediction error
- ▶ Fast since it is embedded in prediction

- **Cons**

- ▶ Does not directly capture the conditional dependence that corresponds to $\tilde{S}$
- ▶ Not naturally suited to hypothesis testing to determine whether a feature exists or not
- ▶ Depends significantly on initial tree splits and often unstable when features are dependent

LOCO (Leave-one-covariate-out)

- **Pros**
- **Cons**

LOCO (Leave-one-covariate-out)

- **Pros**

- ▶ Estimates $\tilde{S}$
- ▶ Captures non-linear dependence
- ▶ Usually gives good prediction error
- ▶ Fast since it is embedded in prediction

- **Cons**

- ▶ Does not directly capture the conditional dependence that corresponds to the standard feature selection criteria
- ▶ Not naturally suited to hypothesis testing to determine whether a feature exists or not
- ▶ Depends significantly on initial tree splits and often unstable when features are dependent

Forward selection AIC/BIC

- **Pros**
- **Cons**

Forward selection AIC/BIC

- **Pros**

- ▶ Captures conditional relationships matching feature selection and $\tilde{S}$
- ▶ Directly imposes sparsity and approximate objectives of interest
- ▶ No need for regularization parameter

- **Cons**

- ▶ Captures only linear relationships
- ▶ Computationally challenging and needs forward selection

Feature selection: Determining feature set

- Select top- k features (e.g. biochemistry example, only k interventions)
 - How do we choose k ?
- Select features above a threshold τ (or let algorithm like Lasso/BIC/AIC choose)
 - How do we choose τ ?
- Use statistical significance with hypothesis testing framework
 - How do we choose size/significance of test?
 - How do we deal with multiple testing issues?

Comparing and validating feature selection methods

- No ground truth with real data so difficult to impossible to validate these
- So far we have focused on statistical arguments related to conditional dependence/independence
- How do we validate and compare feature selection outputs?
- Often preferable to focus on multiple criteria/metrics

Focus on 4 criteria

- Out-of-sample prediction error
- Domain knowledge
- Stability
- Simplicity/interpretability

Recall: Hitters baseball data

- Predicting salary for Baseball players on 1987 (from Kaggle)
- $n = 322$, $p \approx 20$
- 3 "types" of features
 - ▶ Performance stats from previous season (1986): home runs, hits, runs, e.t.c.
 - ▶ Performance stats over career
 - ▶ Other: Years, errors, e.t.c

Hitters - Pre-processing code

```
import numpy as np
import pandas as pd
import itertools
import statsmodels.api as sm
from sklearn.linear_model import LassoCV, LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.utils import resample
from itertools import combinations
```

```
hitters = pd.read_csv("Hitters.csv")

hitters = hitters.dropna()

# Convert categorical → dummies
hitters = pd.get_dummies(hitters, drop_first=True)

# Force EVERYTHING numeric
hitters = hitters.apply(pd.to_numeric, errors='coerce')

# Drop any accidental NaNs
hitters = hitters.dropna()

X = hitters.drop("Salary", axis=1)
y = np.log(hitters["Salary"])

# Final safety cast
X = X.astype(float)
y = y.astype(float)
```

Out-of-sample prediction error

- Default most standard metric
- Can be applied to any algorithm
- Need hold-out/validation data
- Can tend to select a prefer larger feature sets (recall AIC vs BIC)
- Depends not only on the feature set but the type of function

```
# Train/test split  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```


Prediction error for Hitters data - code

- Try it out for AIC, BIC, Lasso, OLS (no p-values) and MDI (simply select top 8 features)

```
# ---- AIC ----
aic_vars = forward_selection(X_train, y_train, "aic")
model_aic = sm.OLS(y_train, sm.add_constant(X_train[aic_vars])).fit()
pred = model_aic.predict(sm.add_constant(X_test[aic_vars]))
results["AIC"] = mean_squared_error(y_test, pred)
selected_features["AIC"] = aic_vars

# ---- BIC ----
bic_vars = forward_selection(X_train, y_train, "bic")
model_bic = sm.OLS(y_train, sm.add_constant(X_train[bic_vars])).fit()
pred = model_bic.predict(sm.add_constant(X_test[bic_vars]))
results["BIC"] = mean_squared_error(y_test, pred)
selected_features["BIC"] = bic_vars

# ---- Lasso ----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = LassoCV(cv=5, random_state=42).fit(X_train_scaled, y_train)
pred = lasso.predict(X_test_scaled)
results["Lasso"] = mean_squared_error(y_test, pred)
selected_features["Lasso"] = list(X.columns[lasso.coef_ != 0])

# ---- Random Forest MDI ----
rf = RandomForestRegressor(n_estimators=500, random_state=42)
rf.fit(X_train, y_train)
pred = rf.predict(X_test)
results["MDI"] = mean_squared_error(y_test, pred)

mdi_importance = pd.Series(rf.feature_importances_, index=X.columns)
mdi_selected = list(mdi_importance.sort_values(ascending=False).head(8).index)
```

Prediction error for Hitters data - results

```
print("Out-of-Sample MSE:")  
for k, v in results.items():  
    print(f"{k}: {v:.2f}")
```

Out-of-Sample MSE:

OLS: 0.46

AIC: 0.45

BIC: 0.47

Lasso: 0.45

MDI: 0.28

- Note MDI significantly better since it uses tree-based ensembles which are a better data fit compared to linear models

Stability

- Stability is a very general concept (we will expand more on it next week)
- Many different types of stability (stability under data perturbations, parameter perturbations, algorithm choices, e.t.c.)
- For feature selection, we can see how stable subsets are under different data perturbations (often slow)
- Many different metrics to compare subset
- One of the most common is Jaccard index

Jaccard index:

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

It follows that $0 \leq J(S_1, S_2) \leq 1$ where $J(S_1, S_2) = 0$ if $S_1 \cap S_2 = \emptyset$ and $J(S_1, S_2) = 1$ if $S_1 = S_2$.

Hitters stability - code

```
def jaccard(set1, set2):
    return len(set1 & set2) / len(set1 | set2)

def bootstrap_stability(method_func, X, y, B=50):
    selections = []
    for _ in range(B):
        Xb, yb = resample(X, y)
        selections.append(set(method_func(Xb, yb)))

    scores = []
    for i in range(len(selections)):
        for j in range(i+1, len(selections)):
            scores.append(jaccard(selections[i], selections[j]))
    return np.mean(scores)

stability = {}

stability["AIC"] = bootstrap_stability(
    lambda Xb, yb: forward_selection(Xb, yb, "aic"),
    X_train, y_train
)

stability["BIC"] = bootstrap_stability(
    lambda Xb, yb: forward_selection(Xb, yb, "bic"),
    X_train, y_train
)

stability["Lasso"] = bootstrap_stability(
    lambda Xb, yb: list(X.columns[
        LassoCV(cv=5).fit(StandardScaler().fit_transform(Xb), yb).coef_ != 0
    ]),
    X_train, y_train
)
```

Model simplicity/interpretability

- As we found with BIC vs AIC, sparser models are sometimes preferred
- Sparser models provide greater interpretability and place a higher bar on feature inclusion than prediction error
- No concrete way to measure other than counting number of features
- If prediction error is not too much higher and we have sparser model (e.g. BIC vs AIC), may prefer sparser model

Domain knowledge

- Any domain knowledge is arguably the most useful validation
- For example in biology/natural science, if you can conduct an experiment to validate the feature set that is often ideal (at times impractical or very difficult)
- In practice, often speak to experts/read papers in that domain to determine features you expect to see
- But don't want to over-index on this since also subjective

Domain knowledge for Hitters

```
: domain_vars = {  
    "Hits", "HmRun", "RBI", "CRuns", "CHits", "CHmRun", "CWalks", "Years"  
}  
  
domain_alignment = {}  
  
for model, vars_sel in selected_features.items():  
    domain_alignment[model] = len(set(vars_sel) & domain_vars)  
  
print(domain_alignment)  
  
{'OLS': 8, 'AIC': 5, 'BIC': 3, 'Lasso': 5, 'MDI': 5}
```

Hitters summery metrics

```
summary = pd.DataFrame({  
    "MSE": results,  
    "Stability": stability,  
    "Model Size": model_size  
})  
  
print(summary)
```

	MSE	Stability	Model Size
OLS	0.459586	NaN	19
AIC	0.450254	0.396192	10
BIC	0.474177	0.296572	3
Lasso	0.450882	0.576513	11
MDI	0.279170	NaN	8

Hitters feature subsets

```
-----  
'AIC': ['CRuns',  
        'Hits',  
        'Years',  
        'League_N',  
        'AtBat',  
        'Walks',  
        'Division_W',  
        'PutOuts',  
        'CWalks',  
        'HmRun'],  
'BIC': ['CRuns', 'Hits', 'Years'],  
'Lasso': ['Hits',  
          'HmRun',  
          'Runs',  
          'Walks',  
          'Years',  
          'CHits',  
          'CRuns',  
          'PutOuts',  
          'Errors',  
          'League_N',  
          'Division_W'],  
'MDI': ['CRuns', 'CAtBat', 'CHits', 'CRBI', 'Hits', 'RBI', 'CHmRun', 'AtBat']}]
```